Compte rendu de réunion n°3 : TER BYZFL

Flexibilité de l'environnement, Industrialisation par JSON et Robustesse aux Seeds

ABBES Samy & AREZKI Célia

22 Mai 2026

Informations générales

— **Date de la réunion :** 22 mai 2026 à 15 h
— **Membres du groupe présents :** ABBES Samy, AREZKI Célia
— **Encadrants présents :** Dupé François-Xavier (en visioconférence), LABOUREL Arnaud
— **Sujet du TER :** Benchmarking et analyse comparative d'algorithmes d'agrégation robustes en apprentissage fédéré face aux adversaires byzantins avec ByzFL

1 Objectifs et Orientations de la réunion

Cette troisième réunion de travail a permis de dresser le bilan de nos scripts initiaux et de poser de nouvelles exigences méthodologiques formulées par nos encadrants. L'accent a été mis sur la flexibilité technique de notre environnement (mise à niveau des dépendances, portabilité matérielle), l'industrialisation rigoureuse de notre protocole d'expérimentation (généralisation par fichiers de configuration) et la validation statistique des résultats.

2 Mise à niveau de l'Environnement et Compatibilité

2.1 Dépendances

La bibliothèque `ByzFL` impose par défaut des versions spécifiques et datées (PyTorch 2.2). Les encadrants nous demandent d'étudier la faisabilité d'une migration vers des versions stables beaucoup plus récentes :

— **PyTorch :** Tenter la mise à niveau vers la version stable actuelle (PyTorch 2.12).
— **Dépendances connexes :** Mettre à jour conjointement Python, NumPy, ainsi que Torchvision afin d'éviter les ruptures de compatibilité.

2.2 Alternative à CUDA pour l'accélération matérielle

L'exécution classique sur GPU via CUDA impose l'utilisation exclusive d'une carte graphique NVIDIA. Or, nos machines de travail actuelles ne disposent pas de ce type de matériel, ce qui empêche d'effectuer les calculs localement avec cette technologie. Les encadrants nous demandent donc de trouver des solutions alternatives pour l'accélération matérielle. Nous devons orienter nos recherches vers l'utilisation de versions plus récentes de PyTorch, qui intègrent de nouveaux frameworks de calcul compatibles avec d'autres architectures matérielles, afin de permettre l'exécution sur nos GPU respectifs.

3 Architecture du Code et Protocoles Expérimentaux

3.1 Paramétrage par fichiers JSON

Pour mettre fin à la duplication de code et garantir une organisation rigoureuse, l'architecture du projet évolue vers un fonctionnement piloté par la donnée. Toutes les expériences seront désormais centralisées et configurées au travers de fichiers de configuration `.json`. Un script d'indexation global permettra de charger dynamiquement ces scénarios sans avoir à modifier manuellement le code source à chaque expérience.

3.2 Suivi de la Convergence et Sauvegarde de l'Accuracy

Afin de pouvoir comparer finement l'efficacité des attaques face aux différents mécanismes de défense, nous devons implémenter une journalisation systématique. L'évolution temporelle de l'accuracy au fil de chaque round d'apprentissage devra être capturée et sauvegardée dans des fichiers `.csv`. Ces métriques permettront de générer des courbes comparatives directes par couple attaque/agrégateur.

3.3 Validation Statistique par Graines Aléatoires (Seeds)

Afin de valider la robustesse de nos conclusions et s'assurer que les performances observées ne relèvent pas du hasard informatique, le protocole intègre des exécutions répétées. Chaque scénario devra être exécuté plusieurs fois en faisant varier systématiquement la graine aléatoire (`seed`), permettant ainsi de moyenner les courbes d'accuracy et d'analyser les écarts-types de convergence.

4 Approfondissement Théorique et Scénarios Critiques

4.1 Exploration fine des hyperparamètres et distributions

Nous devons approfondir l'étude des paramètres internes spécifiques à chaque objet de la bibliothèque tels que :

— **Paramètres d'agrégateurs :** Analyser l'impact de variables clés telles que le facteur de tolérance ou d'échelle τ au sein des filtres.

— **Distribution des données :** Au-delà du cas idéal IID, nous devons explorer la distribution non-IID des données chez les clients en exploitant l'allocation de **Dirichlet** et son coefficient de concentration α.

4.2 Test à la frontière du tiers byzantin ($f = n/3$)

Conformément aux limites théoriques de l'apprentissage fédéré robuste, les encadrants nous demandent de pousser les agrégateurs dans leurs retranchements. Des scénarios spécifiques incorporant exactement ou plus d'un tiers de clients malveillants ($f \geq n/3$) seront programmés afin d'observer expérimentalement l'effondrement ou la résilience des barrières géométriques.

4.3 Diversification des jeux de données

Afin de valider nos benchmarks et s'assurer que nos observations ne sont pas biaisées par la simplicité relative de MNIST, le protocole devra intégrer des tests sur d'autres banques d'images ou jeux de données plus complexes (par exemple Fashion-MNIST ou CIFAR-10).

5 Conclusion et Prochaines étapes

Le plan d'action d'ici la prochaine réunion s'articule autour de trois points :

1. Tester la mise à niveau de l'environnement virtuel avec les versions récentes de Python/PyTorch et valider un *backend* GPU alternatif à CUDA.
2. Finaliser la refonte du code pour migrer vers le système de fichiers de configuration JSON et l'exécution automatisée sur plusieurs *seeds*.
3. Lancer la première campagne de benchmark croisée (Même attaque face à différents agrégateurs) en intégrant les scénarios à forte proportion byzantine et les distributions non-IID via Dirichlet.